

ZMath Shield and ZME1 1.0: Authenticated Evidence Containers with Explicit Security Boundaries

Shafaet Brady Hussain Independent researcher and developer, United Kingdom 18 August 2026

Abstract

This paper specifies and evaluates ZME1, the portable authenticated-container family used by the public ZMath Shield demonstration. The contribution is not a new base cipher. ZME1 composes standard browser cryptography into a versioned JSON envelope: password-based derivation, optional local factors, AES-256-GCM authenticated encryption, canonical authenticated metadata, strict resource bounds, and fail-closed opening. I distinguish three profile identifiers found in the evaluated implementation: a legacy PBKDF2 profile, a PBKDF2-HKDF profile, and an experimental external-factor profile. The term QPU factor in the latter is provenance vocabulary only; the container cannot prove that factor bytes came from quantum hardware and it does not provide post-quantum public-key security.

I evaluated a frozen public implementation snapshot through its file, message, tamper, wrong-factor, resource-bound, known-answer, and simulator-boundary tests. Both test programs completed successfully on 18 August 2026. The release companion records exact SHA-256 hashes, a machine-readable public schema, a structural verifier, and negative test vectors without publishing private key-management or entitlement-service internals. The results support limited claims about envelope behaviour in this snapshot. They do not establish cryptographic certification, endpoint security, independent validation, resistance to offline guessing, remote revocation, or safe production deployment.

Keywords: authenticated encryption, evidence container, AES-GCM, PBKDF2, HKDF, portable file format, provenance, fail-closed design

1. Research question and contribution

The research question is: **Can a protected-file format expose enough public structure for interoperability and falsifiable review while keeping secret material and proprietary policy services outside the publication boundary?**

My answer is a layered specification. The public layer defines parseable fields, authenticated metadata, profile identifiers, input bounds, terminal failures, and reproducible tests. The private layer may manage local vault material, entitlement, recovery, or deployment policy, but those components cannot substitute for standard cryptography or public evidence. This paper contributes:

1. a consolidated public model of the ZME1 family as implemented, rather than the older aspirational file-container draft;
2. an explicit separation between encryption, authentication, policy, transport, and provenance;

3. a claim ledger tied to dated source hashes and executable observations;
4. a threat and misuse analysis, including offline guessing and metadata exposure;
5. a reproducibility companion that checks structural invariants without exposing private production material; and
6. a falsifiable evaluation programme for future independent implementations.

ZMath Shield is the product and policy name. ZME1 is the container family. Neither term denotes a new primitive.

2. Evidence and release boundary

The primary evidence is a local public-web snapshot under the CallChat Shield application, its public profile, a known-answer fixture, and two executable test programs. The evaluated files were hashed before analysis. No credentials, customer files, production containers, account tokens, private ZMath modules, recovery secrets, locked evaluations, or server policy factors are included.

The source snapshot is not a clean Git repository, so I do not claim a commit identity. The release therefore binds its observations to file hashes. The principal implementation file `zshield-core.js` had SHA-256 `c48b5af56a50a2c75f682d56843fdc9a82da7714cda1dde602c5620fa5df8d6`. Its test file had SHA-256 `d184a415cf92d855c481f318f604b861ecbc7d9bba6a76b5414c08a5ad5bd283`. The public known-answer fixture had SHA-256 `ed6a174f387bbc63b82f7b2c2e2c4623a88a481afef28bac4a36b83791173a52`.

The older public profile documents only ZSHIELD-PBKDF2-AESGCM-1. The evaluated implementation additionally accepts ZMATH-PBKDF2-HKDF-AESGCM-2 and ZMATH-PBKDF2-HKDF-AESGCM-QPUFACTOR-3. This paper reports that divergence rather than silently treating documentation and runtime as identical.

3. Container model

Let a container be

$$Z = (H, C)$$

where H is the header object and C is Base64-encoded authenticated ciphertext. The evaluated implementation serialises H with object keys sorted lexicographically at every depth, preserves array order, removes optional whitespace, encodes the result as UTF-8, and supplies those bytes as AES-GCM additional authenticated data.

The header contains:

- `format = "ZME1"` and `version = 1`;
- a recognised profile identifier;
- payload name, media-type hint, byte size, and kind;
- creation time;
- KDF name, iteration count, salts, and factor-required flags;
- cipher name, IV, and tag length; and
- optional bounded context strings for purpose, policy name, transport, key identifier, or evidence references.

The ciphertext encoding appends the 128-bit GCM tag as produced by Web Crypto. Consequently, decoded ciphertext length must equal the authenticated plaintext size plus 16 bytes. The implementation rejects malformed Base64, unsupported identifiers, KDF counts below 600,000 or above 1,200,000, salts other than 16 bytes, IVs other than 12 bytes, payload declarations above 50 MiB, and unsupported tag lengths.

3.1 Authenticated does not mean hidden

Payload name, declared media type, size, kind, timestamp, algorithms, salts, IV, and context are visible. AES-GCM authenticates them because their canonical encoding is additional data, but does not encrypt them. A container can therefore reveal sensitive filenames, approximate content size, timing, application identity, or correlation fields. Implementations should minimise these fields, use neutral display names when necessary, and never put secrets or bearer tokens in context.

3.2 State machine

A conforming opener follows:

```
RECEIVED -> PARSED -> PROFILE_VALIDATED -> FACTORS_CHECKED -> AUTHENTICATED ->
RELEASED
```

Any error transitions to REJECTED, which is terminal. The opener must not return partial plaintext, preview bytes, inferred content, or factor-specific diagnostic detail after an authentication failure. Parsing and resource checks occur before expensive derivation where possible.

4. Cryptographic profiles

4.1 Legacy profile

ZSHIELD-PBKDF2-AESGCM-1 builds input material from UTF-8 passphrase bytes, a zero separator, and either SHA-256 of pattern bytes or 32 zero bytes. PBKDF2-HMAC-SHA-256 with a 16-byte salt and 600,000 iterations derives an AES-256-GCM key. The 12-byte IV is random per container and the tag is 128 bits.

This is an interoperability description, not a proof of security. Concatenating multiple inputs before a password KDF requires careful domain separation and review. Pattern bytes are a second secret input only if they are independently protected and sufficiently unpredictable; a familiar image is not automatically high entropy.

4.2 Current password-plus-pattern profile

ZMATH-PBKDF2-HKDF-AESGCM-2 first derives 256 password-factor bits with PBKDF2-HMAC-SHA-256. It combines a fixed domain string, a separator, that password factor, and a 32-byte pattern digest, then uses HKDF-SHA-256 with a separate 16-byte mixing salt and fixed info string to derive the AES-256-GCM key.

HKDF provides structured extraction/expansion and domain separation; it does not create entropy. If both the passphrase and pattern are guessable or obtained together, the attacker can still test candidates offline against the GCM tag. The implementation requires at least 14 JavaScript Unicode code units in its user interface, but length alone is not a strength measurement. A long unique passphrase generated by a password manager is materially different from a predictable 14-character phrase.

4.3 Experimental external-factor profile

ZMATH-PBKDF2-HKDF-AESGCM-QPUFACTOR-3 adds SHA-256 of supplied external factor bytes to the HKDF input and places the factor digest as a public commitment in the header. A missing or mismatched factor is rejected before decryption.

The label requires caution. At the container boundary, the input is only a byte string. The format cannot establish whether those bytes came from quantum hardware, a simulator, a file, or a deterministic program. A receipt or backend string in authenticated context is a claim carried by the container, not cryptographic attestation. This profile is therefore best described as an **external-factor experiment**. It does not replace a post-quantum key-encapsulation mechanism, resist harvest-now-decrypt-later attacks by itself, or demonstrate quantum advantage.

4.4 Primitive comparison

AES-GCM is a standard authenticated-encryption mode. NIST SP 800-38D specifies GCM and emphasises IV uniqueness. PBKDF2 and HKDF are standard derivation mechanisms available through the W3C Web Cryptography API. The evaluated 600,000 PBKDF2-HMAC-SHA-256 count matches current OWASP guidance for that configuration, but a fixed CPU-bound KDF still requires device-specific latency and denial-of-service testing. OWASP generally prefers Argon2id where available, and RFC 9106 specifies that memory-hard function. A future profile should compare WebAssembly Argon2id or another reviewed memory-hard KDF against browser portability, accessibility, mobile battery cost, and implementation risk.

5. Evidence-container semantics

ZME1 can carry an ordinary file, UTF-8 vault note, Matrix-message envelope, or a future evidence pack. Calling it an evidence container adds obligations beyond encryption:

- a digest identifies the exact payload or external manifest;
- provenance assertions are typed and attributed;
- timestamps distinguish declared creation from independently witnessed time;
- signatures, if later added, bind an identified signing key to canonical bytes;
- verification results distinguish structural validity, authenticated decryption, signature validity, and semantic trust; and
- the viewer never converts a self-asserted context string into independent proof.

Encryption answers who can recover plaintext under the assumed key model. Authentication detects modification relative to the derived key. Neither proves that the plaintext is true, that the author is who they claim, or that a timestamp predates an event. Evidence systems should represent those as separate claims.

6. Evaluation

6.1 Method

On 18 August 2026 I executed the two public Node.js test programs against the frozen files described in Section 2. I did not modify the implementation. The first program exercised file and message creation/opening, the published legacy known-answer vector, canonical authenticated headers, wrong passphrase, wrong pattern, missing pattern, header tampering, malformed IV, excessive KDF work, truncated ciphertext, message-envelope damage, external-factor round trip, missing and wrong external factor, and factor-context tampering. The second program exercised external-factor derivation, evidence fields, tamper handling, and simulator-boundary rules.

The console results were:

```
ZShield file and message round-trip, tamper, factor and resource-bound tests:  
ok
```

```
QPU factor derivation, evidence, tamper and simulator-boundary tests: ok
```

Both processes exited successfully. These are author-run regression tests, not an independent audit. The tests contain multiple assertions but print a single aggregate success line, so I report two passing programs rather than inventing a granular test count.

6.2 What the result supports

The snapshot demonstrates that:

1. the tested legacy vector decrypts to its recorded plaintext;
2. newly created profile-2 file and message containers round-trip in the same implementation;
3. the tested wrong-input and tamper cases fail rather than return plaintext;
4. declared size and KDF bounds are enforced in the tested paths; and
5. profile-3 requires matching external bytes and authenticates tested context fields.

6.3 What the result does not support

The result does not establish cross-language interoperability, nonce uniqueness over population scale, constant-time behaviour, side-channel resistance, browser coverage, secure memory erasure, malware resistance, safe key recovery, entitlement correctness, cryptographic certification, independent validation, or post-quantum security. The code assigns random salts and IVs through Web Crypto, but these tests do not statistically test the random-number generator. They also do not test concurrent generation, rollback, backup leakage, crash recovery, or hostile filenames.

7. Threat model

7.1 In scope

- an attacker obtains a container and attempts offline passphrase guesses;
- fields, ciphertext, or message envelopes are modified;
- an attacker supplies pathological KDF counts, sizes, Base64, or context values;
- a recipient uses the wrong pattern or external-factor file;
- a service or transport observes visible metadata; and
- marketing language incorrectly treats a factor receipt as quantum proof.

7.2 Out of scope or unresolved

- compromised browsers, extensions, operating systems, unlocked endpoints, or keyloggers;
- malicious plaintext processed after successful decryption;
- account takeover, room membership, traffic analysis, and Matrix metadata;
- server-side entitlement and revocation implementation;
- backup, export, escrow, recovery, and multi-device synchronisation;
- coercion, unsafe sharing, or weak human-generated passphrases; and
- formal cryptanalysis or third-party code review.

7.3 Principal risks

Offline guessing. The salt and all verification material required to test a candidate are in the container. The work factor slows guesses but does not prevent them.

Nonce reuse. GCM security depends critically on avoiding IV reuse with the same key. Random 96-bit IVs are appropriate only with a sound generator and lifecycle controls; implementations need collision monitoring assumptions and profile-level limits.

Metadata leakage. Authenticated headers remain visible.

Factor co-location. Storing a pattern or external-factor file beside the container can collapse the intended second-factor benefit.

Canonicalisation drift. Independent implementations must reproduce exactly the same canonical header bytes. JSON number, Unicode, and unsupported-value handling require explicit test vectors.

Resource exhaustion. The 1,200,000-iteration ceiling and 50 MiB payload ceiling reduce some hostile-input costs, but client-side memory and CPU profiling is still required.

Policy overclaim. A live entitlement check can control a compatible client, but cannot reliably revoke plaintext already opened or copied. Portable mode cannot promise remote revocation.

8. Public/private and transport boundaries

ZME1 is additive to transport security. When sent through Matrix, the .zme1 bytes may themselves be carried inside Matrix encrypted attachments. Matrix E2EE protects the room transport and ZME1 protects the portable object under a separate key model. Neither should be presented as replacing device verification, authenticated calling, secure backups, or endpoint controls.

The public format should include algorithm identifiers, limits, canonicalisation, known-answer vectors, threat model, and failure semantics. Private modules may contain entitlement integration, vault storage, UI policy, or commercial implementation details. Secret code is not the security argument: the assurance case rests on reviewed primitives, explicit assumptions, tests, and independent evaluation.

9. Conformance and falsifiability

A **ZME1 structural conformance** claim requires exact parsing rules, recognised profile identifiers, strict Base64 and length checks, canonical header reproduction, and fail-closed rejection. **Cryptographic interoperability** additionally requires known-answer opening across at least two independent implementations. **Operational readiness** additionally requires browser/device matrices, performance budgets, key lifecycle tests, secure recovery decisions, threat-model review, and an external cryptographic assessment.

Future evaluation should freeze:

- at least 100 deterministic positive and negative vectors;
- Unicode normalisation and non-BMP passphrase cases;
- empty, minimum, maximum, and over-limit payloads;
- one-bit mutations across every authenticated field and ciphertext position;
- cross-browser and cross-language implementations;
- PBKDF2 and candidate Argon2id latency distributions on low-, mid-, and high-end devices;
- repeated IV generation with collision-analysis assumptions;
- crash, cancellation, and partial-write behaviour; and
- explicit tests showing that simulator or context labels cannot establish hardware provenance.

The interoperability claim is falsified if a conforming independent implementation cannot open the published vector or generates different canonical bytes. The tamper claim is falsified if any modified authenticated byte produces released plaintext. The factor-separation claim is falsified if a required factor can be omitted without rejection. The resource-bound claim is falsified if hostile declared parameters trigger work beyond the documented ceiling before rejection.

10. Claim ledger

ID	Claim	Evidence	Status
C1	ZME1 uses standard AES-256-GCM rather than a new base cipher.	Frozen source and NIST specification.	Supported for snapshot.
C2	Tested wrong inputs and tampering fail closed.	Author-run public regression program.	Supported only for tested cases.
C3	Metadata is authenticated but visible.	Header supplied as GCM additional data.	Supported by design inspection.
C4	Profile 3 incorporates external factor bytes.	Frozen source and regression test.	Supported for same implementation.
C5	Profile 3 proves quantum-hardware origin.	No attestation evidence.	Rejected.
C6	ZME1 is independently validated or certified.	No independent report or certification.	Rejected.
C7	Portable containers can be remotely revoked after plaintext release.	Contradicts offline-copy model.	Rejected.
C8	The implementation is production secure.	Missing audit, lifecycle, and broad platform evidence.	Not established.

11. Limitations

This is an author-led specification and self-evaluation. I created and maintain related ZMath Shield and CallChat components, which creates a conflict of interest. The evaluated source directory had no available Git identity. File hashes improve snapshot integrity but do not prove provenance or deployment identity. The implementation tests are same-codebase tests; they can share the same misunderstanding. The structural companion deliberately omits encryption code and therefore cannot substitute for a second cryptographic implementation.

The paper does not disclose private key-management or policy internals, but that boundary also prevents public assessment of those components. No formal proof, fuzzing campaign, penetration test, external audit, usability study, accessibility study, or large-scale performance benchmark is reported. No statement here should be interpreted as compliance certification or a guarantee that data cannot be recovered by an attacker.

12. Conclusion

ZME1 is most defensibly presented as a versioned authenticated envelope built from standard primitives, not as a secret or quantum cipher. Its scientific value is the testable separation of visible metadata, derivation, factor mixing, transport, object protection, and evidence claims.

The snapshot passed two public regression programs and rejected the tested tamper, wrong-factor, and excessive-work cases. These results are useful but narrow. The next milestone is an aligned public profile, an independent implementation, memory-hard KDF evaluation, and external cryptographic review.

Data and code availability

The paper, structural schema, conformance verifier, source-hash ledger, and recorded test results are available in the ResearchForumOnline research repository. The release does not include private ZMath modules, credentials, production containers, or server policy material. The public known-answer fixture contains test-only plaintext and credentials and must never be reused in production.

Ethics, privacy, and security

All reported tests used synthetic test strings and public fixtures. No user content or live credentials were processed. Publication separates public verification material from private production secrets. Responsible disclosure should be used for any vulnerability that could expose deployed data.

Conflict of interest

I created and maintain components in the ZMath Shield and CallChat ecosystem. This is not an independent audit. I report negative boundaries and unsupported claims to reduce that bias.

AI-use disclosure

AI-assisted tools were used for corpus search, evidence organisation, code-reading support, test orchestration, and editorial refinement. I am responsible for the released specification, evidence selection, limitations, and claims. Generated prose is not experimental evidence.

References

1. Dworkin, M. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D, 2007. <https://doi.org/10.6028/NIST.SP.800-38D>
2. World Wide Web Consortium. *Web Cryptography Level 2*. <https://www.w3.org/TR/WebCryptoAPI/>
3. Moriarty, K., Kaliski, B., and Rusch, A. *PKCS #5: Password-Based Cryptography Specification Version 2.1*. RFC 8018, 2017. <https://doi.org/10.17487/RFC8018>
4. Krawczyk, H., and Eronen, P. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, 2010. <https://doi.org/10.17487/RFC5869>
5. Biryukov, A., Dinu, D., Khovratovich, D., and Josefsson, S. *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. RFC 9106, 2021. <https://doi.org/10.17487/RFC9106>
6. OWASP Foundation. *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
7. Rescorla, E. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, 2018. <https://doi.org/10.17487/RFC8446>

Appendix A. Snapshot hash ledger

Public artifact	SHA-256
zshield-core.js	c48b5af56a50a2c75f682d56843fdc9a82da7714cda1ddee602c5620fa5df8d6
zshield-core.test.mjs	d184a415cf92d855c481f318f604b861ecbc7d9bba6a76b5414c08a5ad5bd283
zme1-v1.json	ed6a174f387bbc63b82f7b2c2e2c4623a88a481afef28bac4a36b83791173a52

Public artifact	SHA-256
zme1-public-profile-v1.md	7452d675bb735d025ebc99d615055f2644ffe690a6a9dbae6dcb4870691f3333
zmath-public-behaviour-spec.md	60de28c57d61ca40d90f41e6cde2c019bec06bb986c6287b891d5e1339d1ffac

Appendix B. Supersession statement

This paper consolidates and supersedes the generic working paper *ZMath Shield and Portable Evidence Containers: A Public Behaviour Specification* for technical claims about the ZME1 snapshot. It also supersedes the older draft's statement that exact KDF and AEAD choices were entirely deferred. The public runtime now exposes those choices. Private entitlement, recovery, and vault-management internals remain outside the release boundary.